



OPTIMA

AUTOMATED STUDY FLOW

OBJECTIVES CHECKLIST

Julian C · Year 12 Software Engineering
Mentor: Ms West · Client: Sachin Kanagasundaram
Term 2, 2026 · v1.0 · 29 May 2026

Objectives Checklist

A one-to-one trace from each objective specified in the planning documentation to the artefact that satisfies it. Each row has a status badge, the location of this artefact and a one line note on how it was verified.

STATUS LEGEND

- MET** - Implemented, tested, and signed off by the client.
- PARTIAL** - Implemented to a workable level. Improvements have been deferred.
- DEFERRED** - Out of scope for the current version (v1.0) and recorded in the v1.1 backlog.

ORIGINALITY

Optima is my original work and was created for HSC Software Engineering, Term 2, 2026. This includes the Optima application and its documentation files. The project was scoped from a real client interview with the client (Year 12, PMH), who had consented in writing to the use of his identifying information in this submission.

No existing application had been reverse-engineered in the production of Optima. All of the architecture, code, visual design, copy, and documentation are written from scratch. Third-party assets are limited to open-source dependencies (Flask and `pywebview`, each cited under their permissive licence in the Reference manual §13.3) and two SIL Open Font Licence typefaces (Inter and JetBrains Mono).

Where AI assistance was used during development, it was in the role of ‘AI as a client’, used for research (on suitable dependencies, finding resources on coding), draft reviews, and comprehension prompts (to better understand snippets of code online, JavaScript, HTML, CSS, etc.). None was used to generate components, full code units, or full documentation files.

Self-Evaluation

Self-grading every objective as ‘MET’ without any comment would be an empty trace. The summary below reveals the four places where the implementation was weakest. The full evidence tables in §A still record each objective as met; this section’s purpose is to name and identify the gaps in them.

Where	Weakness	Why is it ‘met’ anyway	With more time:
A.4 Priority calculator - urgency override	A hard cutoff at 3 days makes every task in the critical window score 999. An informal piece of homework due tomorrow, and a 30% half-yearly exam due the next day	The override is the planning documentation specification, and the tie-break is deterministic enough for the dashboard. Both evaluators (C and E3) explicitly described the behaviour as a feature instead of a bug, once	Replace the hard cutoff with a continuous urgency curve, modelled using something like $(1 + e^{(-0.5 * days_remaining)})$, so

	rank identically; the tie-break by due date doesn't separate them.	documented.	that within-critical tasks still order by weight.
A.5 Scheduling engine - Greedy, not optimal.	Greedy placement can starve a later task by filling its preferred slot with an earlier one. Has been verified manually. In < 5% of generated weeks, a globally better arrangement exists.	A globally-optimal scheduler would block the UI for seconds on every dashboard load and is NP-hard (non-deterministic polynomial) at scale. The cushion gauge reveals that 'you're overcommitted' when the greedy result is poor, so the user has the signal to adjust.	Add a 'what-if' comparison by running a beam search pass in the background and offering the user the alternative if it successfully dominates the greedy answer during free hours.
A.7 Accessibility - Not externally audited.	WCAG 2.1 AA (Web Content Accessibility Guidelines) compliance is asserted by the developer rather than verified by an external accessibility audit. Some interactive elements may be unreachable by focus.	The four WCAG principles are addressed at the level a single-user offline desktop application warrants (see reference manual §13). The client explicitly validated high-contrast, focus highlights, and zoom.	Run axe-core (accessibility testing engine) via Playwright (headless) against each route, and resolve any AA-level failures.
A.8 Data persistence - No formal migration story.	If a future Optima version adds a field, old account files may not load until their JSON is hand-edited or migrated.	v1.0 is the first version, and there are no old files. Forward compatibility (adding fields and features) is handled because from_dict uses.get(key, default) instead of d[key] on optional fields.	Add a schema_version integer to the JSON, and an explicit migrations module. One numbered upgrade function per breaking change.
SW B – no automated regression suite	Testing is manual and scripted. No pytest suite that runs on every commit exists.	For a 9-module codebase with a single developer and a 5-week timeline, the inner-loop manual smoke test + outer-loop evaluator feedback caught every defect that automated tests would have. See Testing Report §1.	Convert the regression checklist (Testing Report §9 into a Playwright suite that hits every route and asserts on rendered text.
SW D – data structures could be more ambitious	The seven data classes + sorted FreeSlot list cover the problem, but no genuinely novel structures are used (e.g. interval tree for $O(\log n)$ overlap queries).	An interval tree at this scale would be premature optimisation. find_free_slots() measures ~3ms at a realistic 37-event timetable and a full schedule generation stays under 100ms even at an 18x stress test of 666 events (reproducible via tools/benchmark.py; Testing report §7. Bisect-based sorted insertion is the right answer for the actual problem size.	Build an interval tree anyway, for htepractice. Benchmark it head-to-head and document when it would start ot pay off (probably ~500 constraints).

CONTENTS

1. Self-Evaluation (where to push back)
 - A. Functional objectives (from the planning documentation)
 - B. Non-functional objectives
 - C. Rubric requirements
 - D. HSC Software Engineering syllabus tags
 - E. Summary scoreboard.

A. Functional objectives (from planning documentation)

A.1 Authentication and account management

Objective	Status	Evidence	Verification
Sign up with username + password (no email required)	MET ▾	app/auth.py, app/templates/signup.html	Form has no email field. validate_username + validate_password enforce rules.
SHA-256 password hashing with per-user salts.	MET ▾	app/auth.py - hash_password()	Two test users with the same password produce different digests on disk.
Login with error handling for unknown/wrong credentials.	MET ▾	app/main.py - /login route.	Unknown user, wrong password, and blank-field cases all show a typed error message.
Log out to clear the session	MET ▾	app/main.py - /logout route.	Session keys popped; /dashboard redirects back to login.
Multiple accounts coexist on one machine	MET ▾	data/users/<username>.json per account.	Created test and test2; isolated sashboards confirmed by manual test.

A.2 Subject & constraint modelling

Objective	Status	Evidence	Verification
Add/remove subjects with a colour swatch each.	MET ▾	app/templates/preferences.html, /subjects/new, /subjects/<id>/delete	Chip list updates inline, round-trip through JSON is verified.
14-day fortnight timetable as Constraint records.	MET ▾	app/ models.py - constraint dataclass (v1.1: anchor_date + recurrence = 'fortnightly'; day_of_fortnight retained for back-compatibility.	Fortnightly classes are verified to expand correctly across multiple weeks via Constraint.occurs_on(); legacy day_of_fortnight files are

			migrated on first load.
Arbitrary recurrence on fixed events (post-v1.0, from round-4 feedback)	MET ▾	Constraint.recurrence $\in$ {none, daily, weekly, fortnightly, monthly, yearly}; skip_dates, overrides; new editor at /weekly + /api/events	Round-trip has been exercised, created weekly, override one instance, skip another, the scheduler still respects the repeating rule.
Distinguish class periods from study periods and other event kinds.	MET ▾	Constraint.is_study_period + Constraint.kind (subject / extracurricular / appointment/study / other)	Free-period checkbox on the weekly editor's panel; scheduler treats study period as placeable.
Default subjects are seeded on signup.	MET ▾	app/main.py - _seed_default_subjects()	English, Mathematics, Software Engineering, Physics, and Modern History appear on the first login after signing up.

A.3 Assignment lifecycle

Objective	Status	Evidence	Verification
Create an assignment with a subject, title, due date, weighting, and estimated hours.	MET ▾	app/templates/task_form.html, tasks/new	Form validates dates > dtoday and weighting (0-100).
Edit/delete an existing assignment.	MET ▾	/tasks/<id> route handles GET (edit), POST (update), DELETE (remove).	Manual round trip has been confirmed, data persists.
Track progress (0-100%) and adjust without page reload.	MET ▾	app/static/js/app.js, /api/tasks/<id>/progress	Slider value PUT-ed via fetch, dashboard cushion updates on next route hit.

A.4 Priority calculator

Objective	Status	Evidence	Verification
Formula (weighting * 10) / max(days_remaining, 1)	MET ▾	app/templates/task_form.html, tasks/new	Form validates dates > dtoday and weighting (0-100).
Critical override when days_remaining $\leq$ 1	MET ▾	CRITICAL_PRIORITY = 999, URGENCY_THRESHOLD_DAYS = 1	Task due in 2 days outranks every non-critical task on the dashboard.
Tiebreak by due date when scores are equal.	MET ▾	rank_assignments() sorts by (-score, due_date)	Two critical tasks rank earlier due first, matches client request from round 1.

A.5 Scheduling engine

Objective	Status	Evidence	Verification
Auto-generates study blocks across the fortnight.	MET ▾	app/scheduler.py - generate_schedule()	Sample run on the client's real timetable produced a valid week.
Respect 30-120 minute session length with breaks	MET ▾	MIN_SESSION = 0.5, MAX_SESSION = 2.0 BREAK_AFTER_SESSION = 0.25	Generated blocks fall in this band, with a 15-minute break visible between consecutive sessions.
Conflict resolver: never overlap with class periods or existing study blocks.	MET ▾	find_free_slots() subtracts occupied intervals before placement.	The v0.2 overlap bug is fixed in v0.3, verified with the client's timetable.
Cushion gauge: free_hours / workload_hours	MET ▾	calculate_cushion(), doughnut SVG on dashboard.	Adding a long task pushes the gauge to 'low', deleting it restores it.

A.6 Views

Objective	Status	Evidence	Verification
Splash screen	MET ▾	splash.html	Fade-in animation.
Dashboard with KPI strip, priority list, today's schedule, cushion doughnut.	MET ▾	dashboard.html	All four panels visible on the initial load.
Weekly view: 7-day grid, hourly gutter, solid class blocks, dashed study blocks.	MET ▾	weekly.html	Today's column is highlighted, with horizontal scrolling if the user has a narrow window.
Monthly view: calendar grid with assignment dots	MET ▾	monthly.html	Month navigation works, dots are colour-coded by subject.
Preferences page	MET ▾	preferences.html	Theme + accessibility toggles + subject CRUD + constant CRUD all on one page.

A.7 Theming & accessibility

Objective	Status	Evidence	Verification
Light theme (default)	MET ▾	style.css - :root palette	Matches the storyboard palette in the planning document.
Dark theme	MET ▾	[data-theme="dark"] palette.	Toggled live without reload, preserved across logout/login.

High-contrast mode	MET ▾	[data-contrast="high"] palette.	Black text, white background, single accent colour; tested by client in glare.
Focus highlights for keyboard navigation	MET ▾	body.focus=on :focus-visible	3px purple outline on tabbable events.
Zoom level adjustable (1.0-2.0x)	MET ▾	document.body.style.zoom (75-200%), slider + Ctrl/Cmd +/-/0	Number input on Preferences scales every UI element proportionally.

A.8 Data persistence

Objective	Status	Evidence	Verification
One JSON file per user account	MET ▾	app/storage.py	File appears at data/users/<name>.json after signup.
Atomic write to prevent corruption on crash.	MET ▾	tempfile.mkstemp() + os.replace()	Reproduced E3's round-2 crash three times; the files are now intact always afterwards.
All dataclasses round-trip through to_dict / from_dict	MET ▾	app/models.py	Saving/reloading a fully populated account preserves every field.

B. Non-functional objectives

Objective	Status	Notes
Runs offline once installed.	MET ▾	No external HTTP calls in any code path. Verified with Wi-fi off.
Cold-start to the dashboard within a few seconds.	MET ▾	The native window appears within a couple of seconds of python run.py; startup is dominated by the Python interpreter + Flask import + pywebview window creation, not Optima's own code (serverside dashboard render is 1-2ms). Not formally benchmarked.
UI legible at 1280×720	MET ▾	Sidebar collapses to icons below 900px width.
Code organised into a Python package instead of a single file	MET ▾	9 modules organised under app/ (auth, storage, priority, scheduler, models, main, + templates / static / __init__)
No third-party JavaScript framework	MET ▾	Vanilla JS in app/static/js/app.js only.
Suite of documentation matching rubric requirements.	MET ▾	Six documents in this folder, plus a .docx mirror.

C. Rubric requirements

Working software

Requirement	Status	Evidence
Application launches without modification.	MET ▾	pip install -r requirements.txt && python run.py - Installation guide step 5.
GUI implemented (not a CLI-only tool)	MET ▾	pywebview window with HTML/CSS templates.
User input is validated	MET ▾	Username, password, date, weighting and constraint times are all validated server-side; HTML form attributes back this up on the client's side.
Security feature present.	MET ▾	SHA-256 with per-user salt; session-based access control on protected routes.
Complex data structures used appropriately.	MET ▾	Seven dataclasses with composition (User holds Subjects, Constraints, Assignments, ScheduleBlocks, Preferences); scheduler uses a sorted list of FreeSlot intervals.
Application runs without errors during demonstration.	MET ▾	Manual smoke test, all 16 routes respond with 200/302 as expected.

Documentation

Required Document	Status	File
Development Diary	MET ▾	Development Diary.pdf
Evaluations (with user feedback)	MET ▾	Evaluations.pdf
Installation Guide	MET ▾	Installation Guide.pdf
Objectives Checklist	MET ▾	<i>This document.</i>
Reference Manual	MET ▾	Reference Manual.pdf
User Manual	MET ▾	User Manual.pdf

Process

Requirement	Status	Evidence
Iterative development, multiple prototypes.	MET ▾	v0.1, v0.2, v0.3 snapshots preserved in prototypes/
Real user feedback gathered between iterations.	MET ▾	Three feedback rounds were documented in <i>Evaluations</i> .
Feedback acted on and not collected for show.	MET ▾	Each round's 'Synthesis' table maps issues raised → fixes shipped. 14 of 16 raised issues have been fixed.
Project workflow visible	MET ▾	22-entry Development Diary covering Apr 13-May 28.

D. HSC Software Engineering syllabus tags

Where individual outcomes from the Stage 6 Software Engineering syllabus are addressed in the build.

Outcome	How it is addressed
SE-12-01 Applies systems engineering tools.	Iterative prototyping, requirements traced from the planning documentation through evaluations to this checklist.
SE-12-02 Applies tools and resources to design and develop secure code	SHA-256 hashing, per-user salt, session-based access control, atomic JSON writes, server-side input validation.
SE-12-04 Designs software that responds to user needs.	Three feedback rounds shaped the priority formula tie-break, the monthly view, the conflict resolver, and the high-contrast mode.
SE-12-08 Tests and evaluates the suitability of a software solution.	Smoke tests on every prototype release, an evaluation panel of three end users plus the client, addendum-style change log in <i>Evaluations</i> .

E. Summary scoreboard

Section	Items	MET ▾	PARTIAL ▾	DEFERRED ▾
A. Functional	30	30	0	0
B. Non-Functional	6	6	0	0
C. Rubric requirements	15	15	0	0
D. Syllabus Tags	4	4	0	0
Total	55	55	0	0

Acknowledged v1.1 backlog

Items deliberately not implemented for v1.0, so they are not lost.

- + **XP counter:** The counter increments on task completion, but has no effect and is purely aesthetic currently. To be turned into a gamification spec in future versions to motivate users.
- + **Calendar export:** Not asked for in planning, but would be a needed addition for future versions (.ics files of generated study blocks for import into Apple Calendar / Google Calendar).
- + **Cross-device sync.** Out of scope by design. The brief specified an offline desktop app.